



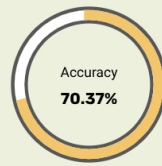
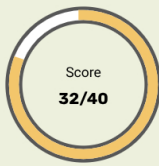
Could not check previous requests.



Well done! You did well on this assessment.

[Request Re-evaluation](#)

Submitted on 28 Aug 2026, 9:28 AM



Your Responses

Question 1 of 27

1/1

A system receives already-sorted input and always selects the last element as the pivot. Why is this dangerous?

The pivot is always near the middle

Every partition can produce a 1 vs n-1 split ✓

Partition becomes $O(1)$

The array becomes automatically sorted without recursion

[Show Explanation](#) ▼

Question 2 of 27

1/1

Merge Sort has the recurrence $T(n) = 2T(n/2) + O(n)$. What determines the Master Theorem case?

Whether the input contains duplicates

Comparing $O(n)$ with $n^{\log_2 2}$ ✓

Whether the array is already sorted

Whether the recursion has one or two base cases

[Show Explanation](#) ▼

A student claims: "Since all elements before the pivot are greater than it, the pivot must land at index 0." What did the student miss?

j never scans the pivot

Elements equal to the pivot also satisfy

`A[j] <= pivot`



i should start at 0

The pivot is never swapped

Show Explanation

Suppose a Quick Sort receives an input where every partition is perfectly balanced. How does its asymptotic time compare with Merge Sort?

Quick Sort becomes $\Theta(n^2)$ while Merge Sort is $\Theta(n)$

Both are $\Theta(n \log n)$



Quick Sort is $\Theta(\log n)$ while Merge Sort is $\Theta(n \log n)$



Quick Sort is always slower

Show Explanation

A Java implementation uses `int mid = (lo + hi) / 2` for a very large sorted array. What is the safest replacement?

`mid = (lo + hi) * 2`

`mid = (hi - lo) / 2`

`mid = lo + (hi - lo) / 2`



`mid = hi - (lo + hi) / 2`

Show Explanation ▾

Question 6 of 27

1/1

Which three properties must a recursive algorithm satisfy to follow the Divide and Conquer approach?

Independent subproblems, balanced division, and a well-defined combine step ✓

Overlapping subproblems, sequential processing, and recursion

A single subproblem, unequal division, and no combine step

Repeated computation, balanced division, and no combine step

Show Explanation ▾

Question 7 of 27

1/1

A sorted array is used to find any occurrence of a value that appears several times. The predicate `nums[mid] == target` can change from false to true and then back to false. What should be done to make binary search applicable?

Use a monotonic predicate such as `nums[mid] >= target` ✓

Remove the sorting requirement

Use `mid = (lo + hi) / 2` instead

Continue using the equality predicate because duplicates do not matter

Show Explanation ▾

Question 8 of 27

1/1

Why does Master Theorem apply to Quick Sort's balanced recurrence but not its worst-case recurrence?

The worst case has no recursion

Balanced Quick Sort uses subproblems of size n/b , while the worst case uses $n-1$ ✓

Master Theorem only applies to Merge Sort

Worst-case Quick Sort has constant work

Show Explanation ▾

Question 9 of 27

1/1

A programmer changes the update $hi = mid - 1$ to $hi = mid$. On some inputs, the search keeps repeating the same range. What is the reason?

The array becomes unsorted after the update

The midpoint calculation causes integer overflow

The loop always checks one element twice

The search space may fail to shrink because mid remains inside the range ✓

Show Explanation ▾

Question 10 of 27

1/1

After Lomuto partition finishes, what is guaranteed about the pivot?

It is the smallest element

It is at its final sorted position ✓

The entire array is sorted

Both recursive halves are already sorted

Show Explanation ▾

A student uses Quick Sort to arrange a list of marks. They choose the middle element as the pivot. Can Quick Sort still take a long time for some inputs?

No, it will always be fast

Yes, some inputs can still lead to poor performance ✓

It becomes Merge Sort

It always takes $O(n \log n)$

[Show Explanation](#) ▼

A developer needs to sort data and has limited extra memory. Stability is not important. Which sorting method is a suitable choice?

Merge Sort

Quick Sort ✓

Bubble Sort

Selection Sort ✗

[Show Explanation](#) ▼

Why does brute-force inversion counting take $O(n^2)$ time?

It recursively sorts the array twice

It checks every possible pair of elements ✓

It performs n merge operations

It uses an additional $O(n)$ array

Show Explanation ▾

Question 14 of 27

0/1

A developer changes if `L[i] <= R[j]` to if `L[i] < R[j]` in inversion-counting Merge Sort. What problem can this cause when duplicate values exist?

It always makes the algorithm

`O(n2)`



Equal elements can be counted as inversions



It prevents the array from being split

It causes the recursion to stop early

Show Explanation ▾

Question 15 of 27

1/1

What is the key advantage of counting inversions during Merge Sort instead of using a separate pair-checking pass?

It changes Merge Sort to $O(n^2)$

It eliminates recursion completely

It counts multiple inversions at once during the existing $O(n)$ merge



It requires sorting the array again

Show Explanation ▾

Question 16 of 27

0/1

A security-sensitive application cannot tolerate $\Theta(n^2)$ worst-case behavior, but available memory is limited. Which algorithm best fits the lesson's recommendation?

Merge Sort



Pure deterministic Quick Sort

Introsort ✓

Bubble Sort

Show Explanation ▾

Question 17 of 27

1/1

Which statement correctly describes the three clauses of the binary search loop invariant?

The array is sorted, the search space remains non-empty, and the target, if present, remains within $[lo, hi]$ ✓

The array is sorted, mid always equals the target, and both pointers move every iteration

The array is unsorted, lo is always smaller than mid, and the target is always at mid

The array is sorted, lo and hi always differ, and the target must be outside the discarded half

Show Explanation ▾

Question 18 of 27

1/1

A recurrence has the form $T(n) = 2T(n/2) + O(n)$. What do the values $a = 2$, $b = 2$, and $f(n) = O(n)$ represent?

One subproblem, size $n/2$, and constant combine cost

Two independent subproblems, each of size $n/2$, and an $O(n)$ combine cost ✓

Two subproblems, each of size n , and an $O(n)$ combine cost

Two overlapping subproblems, each of size $n/2$, and an $O(1)$ combine cost

Show Explanation ▾

Question 19 of 27

0/1

Why is the merge step of Merge Sort $O(n)$?

Each element is compared with every other element

The two halves are recursively sorted again ✗

Every element is placed into the output once, with at most one comparison per placement ✓

The midpoint calculation takes $O(n)$

Show Explanation ▾

Question 20 of 27

1/1

Which statement best captures the structural difference between Merge Sort and Quick Sort?

Both perform their main work during merging

Merge Sort works during merge; Quick Sort works during partition ✓

Quick Sort has a combine step while Merge Sort does not

Both perform all work after recursion

Show Explanation ▾

Question 21 of 27

0/1

For the worst-case recurrence $T(n) = T(n-1) + O(n)$, what is the main reason the complexity becomes $\Theta(n^2)$?

There are two recursive calls at every level

The work forms the sum $n + (n-1) + (n-2) + \dots + 1$ ✓

Partition takes $O(n^2)$ by itself ✗

The pivot selection takes $O(n^2)$

[Show Explanation](#) ▾

Question 22 of 27

1/1

Why doesn't randomized Quick Sort completely eliminate the worst case?

Randomization makes every partition balanced

Randomization removes recursion

A particular sequence of random choices can still produce $\Theta(n^2)$ behavior ✓

Randomized Quick Sort always uses Merge Sort

[Show Explanation](#) ▾

Question 23 of 27

1/1

A binary search reaches $lo = hi = 5$, and `nums[5]` is the target. Why must the loop still execute at this point?

Because the midpoint cannot be calculated when $lo = hi$

Because `lo <= hi` keeps the single remaining element inside the search space ✓

Because the algorithm must always perform two more iterations

Because the array is no longer sorted when only one element remains

[Show Explanation](#) ▾

Question 24 of 27

0/1

A Quick Sort implementation repeatedly chooses the smallest element as the pivot. What happens to the recursion tree?

- It becomes logarithmic in depth ✕
- Each level creates two equal subproblems
- It collapses into a chain of depth n ✓
- Recursion stops after one level

Show Explanation ▾

Question 25 of 27

0/1

A leaderboard must preserve the original order of players who have equal scores. Which algorithm should be preferred?

- Quick Sort ✕
- Introsort
- Merge Sort ✓
- Selection Sort

Show Explanation ▾

Question 26 of 27

5/5

You own an online bookstore and have a list of book prices in random order. Your goal is to sort them from lowest to highest so customers can easily browse. Use Quick Sort to achieve efficient sorting.

```
FUNCTION quickSort(array, low, high)
  IF low < high THEN
    pivotIndex <- partition(array, low, high)
    quickSort(array, low, pivotIndex - 1)
    quickSort(array, pivotIndex + 1, high)
  END IF
END FUNCTION

FUNCTION partition(array, low, high)
  // Select the last element as the pivot
  pivot <- array[high]

  // i is the index for the boundary of the smaller elements partition
  i <- low - 1

  FOR j FROM low TO high - 1
    // If current element is smaller than or equal to the pivot
    IF array[j] <= pivot THEN
      i <- i + 1
      SWAP array[i] WITH array[j]
    END IF
  END FOR
```

```
// Place the pivot in its correct final position
SWAP array[i + 1] WITH array[high]

RETURN i + 1
END FUNCTION
```

1/5

In this bookstore example, what does the pivot element do?

Finds the cheapest book

Divides the list into cheaper and costlier books ✓

Removes duplicate prices

Calculates the average price

2/5

What is the average time complexity of Quick Sort?

$O(n^2)$

$O(\log n)$

$O(n)$

$O(n \log n)$ ✓

3/5

In the partition method, why is the index i initialized to `low - 1`?

To avoid comparing the first element of the sub-array.

To correctly establish the boundary for elements smaller than the pivot. ✓

To skip over any duplicate elements in the array.

To reduce the total number of swaps required by half.

4/5

If the quickSort method is called with the array `bookPrices = {750, 350, 150, 999}`, what will be the value of the pivot in the first partition call?

150

350

999	✓
750	

5/5

If the quickSort method is called on the array `{499, 199, 999, 150}`, what will the final sorted array be?

199 150 499 999	
150 199 499 999	✓
999 499 199 150	
150 499 199 999	

Question 27 of 27

10/10

A logistics company receives packages throughout the day, each assigned a unique priority score. Before dispatch, the packages must be arranged in ascending order of their priority scores so that the dispatch system can process them efficiently.

Given the priority scores of **N** packages, sort them in ascending order and print the sorted sequence.

Note: Built-in sorting functions are not allowed.

Input Format:

- The first line contains an integer **N**, representing the number of packages.
- The second line contains **N** space-separated integers representing the priority scores.

Output Format:

Print the package priority scores in ascending order, separated by spaces.

Example 1:

Input: 6 45 12 78 23 9 56 Output: 9 12 23 45 56 78
--

Explanation:

After sorting the package priority scores in ascending order, the dispatch queue becomes: 9, 12, 23, 45, 56, 78.

Example 2:

Input: 5 100 50 100 25 75 Output: 25 50 75 100 100
--

Explanation:

The priority scores are arranged in ascending order while preserving all occurrences of duplicate values.

Example 3:

```
Input:
7
-5 12 0 -10 8 3 -2

Output:
-10 -5 -2 0 3 8 12
```

Explanation:

The package priority scores include both negative and positive values. After sorting them in ascending order, the correct dispatch order is obtained.

Constraints:

- $1 \leq N \leq 200000$
- $-10^9 \leq \text{Priority Score} \leq 10^9$
- Built-in sorting methods (such as `sort()`, `Arrays.sort()`, `Collections.sort()` etc.) are **not allowed**.
- Your solution should be efficient enough to handle the given constraints.
- Time complexity: $O(n \log n)$ average case (Quick Sort), $O(n^2)$ worst case
- Space complexity: $O(\log n)$ average recursion stack (Quick Sort)

JAVA

```
1 import java.util.Scanner;
2
3 public class Solution {
4
5     static void quickSort(int[] arr, int low, int high) {
6         if(low<high){
7             int pi = partition(arr,low,high);
8             quickSort(arr,low,pi-1);
9             quickSort(arr,pi+1,high);
10        }
11    }
12
13    static int partition(int[] arr,int low , int high ){
14        int pivot = arr[high];
15        int i = low-1;
16        for(int j=low;j<high;j++){
17            if(arr[j]<=pivot){
18                i++;
19                int temp = arr[i];
20                arr[i] = arr[j];
21                arr[j] = temp;
22            }
23        }
24        int temp = arr[i+1];
25        arr[i+1] = arr[high];
26        arr[high] = temp;
27
28        return i+1;
29    }
29 }
```

RunRun TestsChecks

